

Travel Log

System Design Document

Tento systémový návrh detailně popisuje technickou architekturu, datový model a bezpečnostní strukturu aplikace Travel Log. Dokumentace se zaměřuje na integraci backendového řešení ve frameworku Spring Boot s perzistentní MySQL databází, implementaci bezpečné autorizace pomocí JWT tokenů a komponentovou strukturu frontendové části v knihovně React.

Obsah

Architektura aplikace	2
ADR	2
UML Deployment Diagram	3
Popis API	4
Administrátorské endpointy	4
Databáze	5
Entity-Relationship Diagram	5
Popis databázových tabulek.....	5
Návrh uživatelského rozhraní.....	6
Design systém a CSS proměnné.....	6
Barevná paleta	6
Typografie	6
Vizuální komponenty a rozvržení systému.....	7
Náhled návrhu uživatelského rozhraní	7
Závěr.....	8

Poznámka k dokumentaci: vzhledem k vysoké komplexnosti architektonického návrhu, datového modelu a uživatelského rozhraní je součástí projektové dokumentace samostatný soubor s názvem „**Kompletní dokumentace**“. Tento dokument obsahuje kompletní schémata i finální prototypy rozhraní v plném rozlišení pro detailní přezkoumání.

Architektura aplikace

ADR

Backend: Výběr frameworku

Kontext	Je potřeba vytvořit stabilní serverovou aplikaci, která bude spravovat byznys logiku a bezpečně komunikovat s databází
Rozhodnutí	Použití Spring Boot 3.3.5 s Java 17
Důvod	Framework nabízí skvělou integraci se Spring Security a Spring Data JPA. Umožňuje rychlý vývoj REST API pomocí anotací.
Důsledky	Aplikace bude rozdělena do vrstev (Controller, Service, Repository), což usnadňuje testování a údržbu.

Databáze: Objektově-relační mapování

Kontext	Aplikace využívá pro perzistenci dat relační databázi MySQL umístěnou na serveru. Je nutné tento relační svět efektivně propojit s objektovým světem Javy bez psaní nadměrného množství SQL dotazů.
Rozhodnutí	Relační databázový server MySQL v kombinaci se standardem Spring Data JPA.
Důvod	MySQL je stabilní, produkčně ověřený databázový stroj. Spring Data JPA nad ním tvoří abstrakci, která automaticky generuje dotazy (ve správném MySQL dialektu), bezpečně spravuje připojení na server a řeší relační integritu.
Důsledky	Přístup k databázi je plně objektový. Je nutné správně definovat kardinality (např. @ManyToOne) a využívat líné načítání (FetchType.LAZY), aby se šetřil síťový výkon při dotazování na server.

Frontend: Správa stavu a komunikace

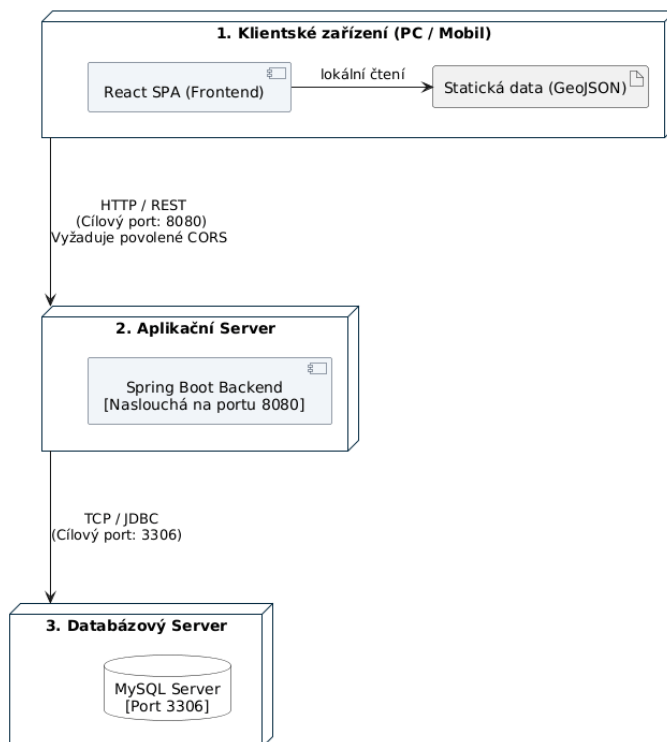
Kontext	Aplikace vyžaduje interaktivní uživatelské rozhraní (klikací mapa), které okamžitě reaguje na akce. Zároveň je nutné efektivně zobrazovat statická encyklopedická data a jejich překlady bez zbytečného zatěžování backendu.
Rozhodnutí	Využití knihovny React pro UI, standardního Fetch API pro asynchronní komunikaci s backendem a lokálních JSON souborů pro překlady a statická data.
Důvod	React díky virtuálnímu DOMu zajišťuje okamžité a efektivní překreslování komponent. Přesun statických dat do lokálních JSON souborů radikálně snižuje latenci a zátěž databázového serveru.
Důsledky	Frontend je architektonicky plně oddělený od backendu. Toto oddělení vyžaduje striktní nastavení CORS (Cross-Origin Resource Sharing) na straně Spring Bootu, aby prohlížeč komunikaci mezi doménami nezablokoval.

Bezpečnost: Autentizace pomocí JWT

Kontext	Aplikace vyžaduje bezpečný přístup k osobním datům uživatele. Vzhledem k oddělenému frontendu a backendu je nutné zvolit mechanismus, který je bez stavový a bezpečný proti běžným útokům.
Rozhodnutí	JSON Web Token (JWT) v kombinaci se Spring Security.
Důvod	Bez stavovost, Standardizace, Bezpečnost
Důsledky	Hesla budou v DB uložena jako hashe (BCrypt). Každý request z frontendu musí obsahovat token, jinak backend vrátí chybu.

UML Deployment Diagram

1. Klientské zařízení (Frontend)
 - Zastupuje webový prohlížeč na koncovém zařízení (např. PC)
2. Aplikační server
 - Java Runtime Environment, kde běží Spring Boot backend naslouchající na portu 8080
3. Databázový server
 - Relační databázový stroj MySQL, který spravuje databázi a naslouchá na portu 3306



Popis API

Základní URL: `http://localhost:8080/api` (lokální vývojové prostředí).

Formát dat: Výměna dat probíhá obousměrně ve formátu `application/json`.

Zabezpečení: Všechny chráněné endpointy vyžadují přítomnost validního přístupového tokenu v http hlavičce.

Administrátorské endpointy

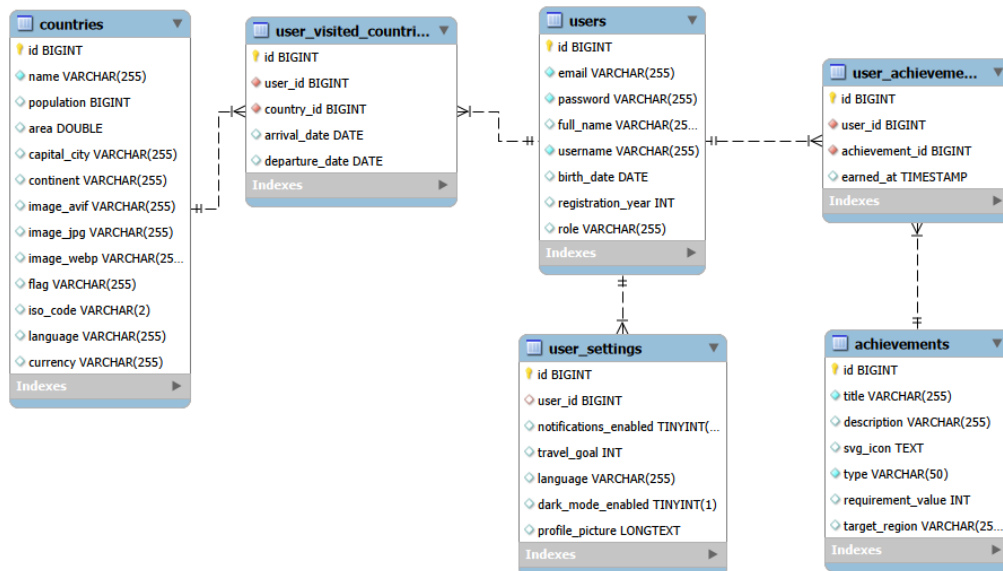
HTTP Metoda	Endpoint	Popis funkce	Zabezpečení
POST	<code>/api/users/register</code>	Registrace nového účtu.	NE
POST	<code>/api/users/login</code>	Přihlášení uživatele a vystavení přístupového JWT tokenu.	NE
GET	<code>/api/countries</code>	Načtení kompletního seznamu všech států z databáze.	NE
GET	<code>/api/countries/{name}</code>	Vyhledání a načtení detailů konkrétní země.	NE
GET	<code>/api/settings</code>	Načtení uživatelského nastavení.	ANO (user)
PUT	<code>/api/settings/update</code>	Aktualizace uživatelského nastavení.	ANO (user)
GET	<code>/api/achievements</code>	Načtení achievementů pro aktuálního uživatele.	ANO (user)
GET	<code>/api/visits/my-visits</code>	Získání kompletní historie cest aktuálního uživatele.	ANO (user)
POST	<code>/api/visits/add</code>	Vytvoření nového záznamu o navštívené zemi.	ANO (user)
PUT	<code>/api/visits/{id}</code>	Úprava existujícího záznamu o cestě.	ANO (user)
DELETE	<code>/api/visits/{id}</code>	Odstranění záznamu o cestě.	ANO (user)
GET	<code>/api/users</code>	Administrátorský výpis kompletního seznamu všech registrovaných uživatelů.	ANO (admin)
GET	<code>/api/users/stats/summary</code>	Vygenerování celkových statistik systému	ANO (admin)
PUT	<code>/api/users/{id}/role</code>	Změna uživatelské role	ANO (admin)
DELETE	<code>/api/users/{id}</code>	Trvalé odstranění uživatelského účtu včetně všech jeho dat z databáze.	ANO (admin)

Databáze

Data jsou ukládána v databázi MySQL 8.0

Entity-Relationship Diagram

Tento diagram reprezentuje fyzický datový model. Zachycuje tabulky, jejich sloupce, datové typy, primární a cizí klíče včetně kardinality.



Popis databázových tabulek

Fyzický model se skládá ze šesti tabulek, které lze rozdělit na jádro systému (uživatelská a encyklopedická data) a subsystém herních prvků (achievementsy).

1. **users**
 - Uchovává základní přihlašovací a profilové údaje jako je e-mail, zašifrované heslo, uživatelské jméno a přístupová role.
2. **countries**
 - Obsahuje encyklopedická data o státech světa a textové cesty k obrázkům a vlajkám na frontendu.
3. **user_visited_countries**
 - Zaznamenává samotné výlety. Propojuje konkrétního uživatele s navštívenou zemí a ukládá přesné datum příjezdu a odjezdu.
4. **user_settings**
 - Ukládá osobní předvolby každého uživatele, jako je zapnutí tmavého režimu, preferovaný jazyk aplikace a osobní cestovatelský cíl.
5. **achievements**
 - Definuje všechny dostupné cestovatelské výzvy a úspěchy, které lze v aplikaci odemknout (obsahuje jejich název, popis, podmínku pro splnění a ikonu).
6. **user_achievements**
 - Propojuje uživatele s odznaky, které již úspěšně získal, a eviduje čas jejich odemčení.

Návrh uživatelského rozhraní

Desing aplikace je navržen s důrazem na minimalismus a intuitivní ergonomii. Vizuální styl kombinuje styl moderní aplikace s herními prvky.

Design systém a CSS proměnné

Pro zajištění konzistence napříč aplikací byly implementovány proměnné pomocí SCSS preprocesoru. Tento systém definuje pravidla pro barvy, typografii a tvary prvků, které jsou následně mapovány na CSS proměnné, což umožňuje plynulé přepínání tmavého režimu.

Barevná paleta

Použité barvy

- #042940 - Primární barva
- #2BC3FF - Akcentní barva
- #343A40 - Barva textu
- #FAFAFA - Pozadí stránky
- #CCCCCC - Výplň nesplněných cílů
- #FFFFFF - Pozadí karet
- #ce0306 - Chybové stavy
- #009C05 - Indikace úspěchu

Použité barvy - tmavá verze

- #042940 - Primární barva
- #2BC3FF - Akcentní barva
- #FFFFFF - Barva textu
- #2D2D2D - Pozadí stránky
- #555555 - Výplň nesplněných cílů
- #373737 - Pozadí karet
- #ce0306 - Chybové stavy
- #009C05 - Indikace úspěchu

Interaktivní prvky (tlačítka, odkazy...) pracují se stavy hover a disable, čehož je docíleno změnou velikosti, stínování a jiným nastavením jasu nebo průhlednosti použitých barev.

Typografie

Typografický systém je navrhnut s ohledem na maximální čitelnost a kontrast aplikace. V návrhu aplikace jsou využívány dva styly písem pro dosažení optimálního kontrastu.

Styl písma

Běžný text: Helvetica

Text pro nadpisy: Montserrat

Letter spacing

Text pro nadpisy: -1px

Text pro tlačítka: 2px

Velikost písma

Základní: 16px

Zlatý řez: 1,618

Pro dosažení co nejlepší vizuální harmonie je využívám zlatý řez, kterým se násobí případně dělí základní písmo dle potřeby.

Vizuální komponenty a rozvržení systému

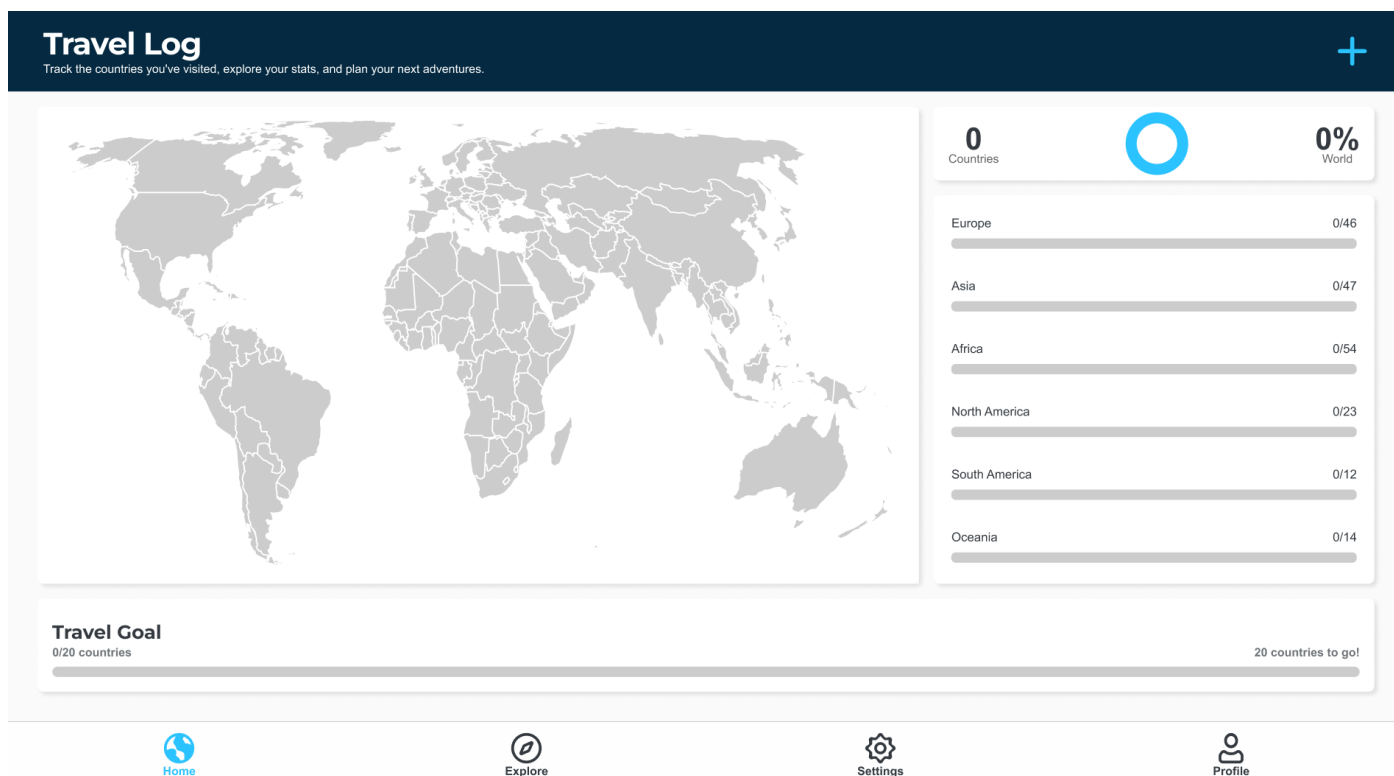
Zaoblení hran: Všechny obdélníkové prvky využívají zaoblení hran 8px. Vyjimku tvoří prvky na úvodní stránce, kde je zaoblení u pozadí a ohraničení sekcí, kde je zaoblení výraznější a to 30px.

Stínování: K vytvoření dojmu hloubky se využívají stíny (4px 4px 6px rgba(0, 0, 0, 0.1)), které jsou u karet a tlačítek).

Ikony: Pro maximalizaci uživatelského zážitku a efektivní využití prostoru byly do návrhu UI začleněny čisté, lineární SVG ikony. Tyto ikony slouží jako primární vizuální navigace v menu a jako doplňková orientace uvnitř jednotlivých komponent. Tento přístup výrazně minimalizuje nutnost použití textových popisků, což je klíčové zejména při zobrazení na mobilních zařízeních.

Náhled návrhu uživatelského rozhraní

Níže uvedený vizuální výřez představuje reprezentativní náhled uživatelského rozhraní hlavní obrazovky systému (Dashboardu).



Vzhledem k rozsahu návrhu je součástí dokumentace soubor s názvem „**Kompletní dokumentace**“, který obsahuje kompletní návrh uživatelského rozhraní.

Závěr

Předložený architektonický návrh prokazuje, že systém Travel Log je navržen na pevných, moderních a stabilních základech. Zvolené propojení backendového řešení ve Spring Bootu s frontendovou částí v Reactu poskytuje aplikaci potřebnou robustnost, bezpečné nakládání s uživatelskými daty a vysokou škálovatelnost. Definovaná komponentová architektura a datový model jsou zároveň plně připraveny na případné budoucí rozšiřování, ať už se bude jednat o integraci dalších externích služeb, nebo o striktní optimalizace výkonu a plynulosti samotného uživatelského rozhraní.